

GESTIONE DEL PROGETTO

Software Life Cycle

Processo di sviluppo: Agile Scrum

In conformità con quanto definito nel project plan , il team ha adottato un modello di progetto Agile ispirato a SCRUM, applicandolo in maniera più leggera per adattarsi alle esigenze del team.

Il ciclo di vita è stato strutturato in sprint, preceduti da una fase preliminare.

- Sprint 0: fase di prototipazione per comprendere tecnologie e architettura base
- Sprint 1-6: sviluppo delle funzionalità

Ogni sprint era preceduto da uno sprint planning e seguito da una sprint review (ed eventualmente una sprint retrospective).

Durante la fase di esecuzione il team si aggiornava tramite Daily Scrum in presenza o asincrone online o tramite il gestore di configurazione.

Differenze con il project plan

Pianificato: la durata dello sprint era di 10 giorni circa

Effettivo: a causa di sessioni di esame la durata degli Sprint è diventata maggiormente flessibile, circa da 10 a 20 giorni in base al periodo. Questo adattamento è stato necessario e rispecchia la filosofia SCRUM riguardante il cambiamento.

La gestione degli sprint è stata la seguente (le stesse informazioni sono ritrovabili all'interno del file Meeting_Sprint):

Sprint 0 (Setup e prototipazione): 15/10 - 02/11 (Analisi, Scelta Tecnologie)

Sprint 1 (Core): 02/11 - 10/11 (Login, Registrazione)

Sprint 2 (Mappa e database): 10/11 - 19/11 (Mappa, Integrazione Firebase)

Sprint 3 (Logica prenotazioni e controlli): 19/11 - 27/11 (Prenotazioni, Logica business)

Sprint 4 (QR e garage): 28/11 - 10/12 (QR Code, Auto, Refactoring)

Sprint 5 (Funzionalità secondarie): 11/12 - 30/12 (Admin Dashboard, Geolocalizzazione)

Sprint 6 (Finale): Gennaio (Documentazione, Chiusura)

Approccio MDA

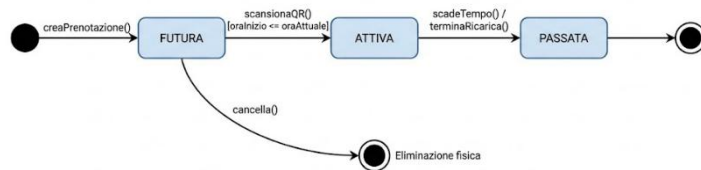
È stato utilizzato inizialmente un approccio MDA con stesura di diagrammi dei casi d'uso e della macchina a stati utili per definire i confini del sistema, le funzionalità dell'utente e il ciclo di vita dell'applicazione, dall'apertura alla conferma della prenotazione.

Essi sono stati utili poi a ricostruire il codice del sistema e a comprendere in maniera più chiara le interazioni tra gli attori.

Qui un esempio di diagramma degli stati della prenotazione, implementato poi all'interno del codice.

Stati possibili:

- Futura
- Attiva
- Passata



Configuration Management

Strumenti e Workflow

Il team ha utilizzato GitHub come gestore della configurazione.

Repository: È stata utilizzata una repository privata, accessibile ai membri del gruppo e ai professori.

Branch: sono stati creati ed utilizzati nove branch, per gestire modifiche complesse o per prove su cambiamenti interni al sistema.

Tutti i commit poi sono stati inseriti nel main branch, che contiene la versione stabile, corretta e finale del progetto.

Comandi git utilizzati

Le operazioni venivano gestite tramite GitHub desktop, questo ci permetteva di utilizzare i comandi principali senza doverli esplicitare direttamente.

Il flusso di modifica di codice era il seguente:

- Apertura GitHub desktop
- Fetch e pull del progetto (git fetch, git pull)
- Modifiche in Eclipse e salvataggio
- Creazione e invio commit con titolo e descrizione (git add, git commit -, "...")
- Push del commit sulla repo (git push)

Gestione delle attività

Per l'organizzazione del lavoro sono stati utilizzati i seguenti strumenti:

- 1- Issues: svolgevano il ruolo di product backlog. Ogni requisito mancante o bug veniva aggiunto dal membro del team (in particolare dal PO) con annessa descrizione e label

Qui un esempio (in data 10/10/2026):

☐ Open 10	Closed 24	Author	Labels	Projects	Milestones	Assignees	🔍 Newest
☐	☒						
Modificare state machine diagram #39 · dmascheretti opened 44 minutes ago							
☐	☒						
Package diagram #38 · dmascheretti opened 16 hours ago							
☐	☒						
Component diagram #37 · dmascheretti opened 16 hours ago							
☐	☒		enhancement	invalid			👤
Rivedere la sidebar per l'admin #36 · Tommaso-Maistrello opened yesterday							
☐	☒						👤
Divisione CSS da Java #31 · Tommaso-Maistrello opened 2 weeks ago							
☐	☒		enhancement				👤
Dividere RegisterView in metodi #27 · dmascheretti opened 3 weeks ago							
☐	☒		enhancement				👤
Sistemare dialog nuova colonnina in AdminColonnineView #24 · Tommaso-Maistrello opened on Dec 11, 2025							
☐	☒		enhancement				👤
Aggiungere ricerca luoghi sopra la mappa #22 · dmascheretti opened on Dec 10, 2025							
☐	☒		enhancement				👤
Separazione della generazione della mappa rispetto alla lettura delle colonnine #8 · Tommaso-Maistrello opened on Nov 18, 2025							
☐	☒		enhancement				👤 1
Implementazione visualizzazione stato carica auto #5 · dmascheretti opened on Nov 18, 2025							

- 2- Project Board: il team ha utilizzato la kanban board automatica di GitHub, messa con visibilità privata.

Si distinguevano 3 categorie: To Do, In Progress, Done.

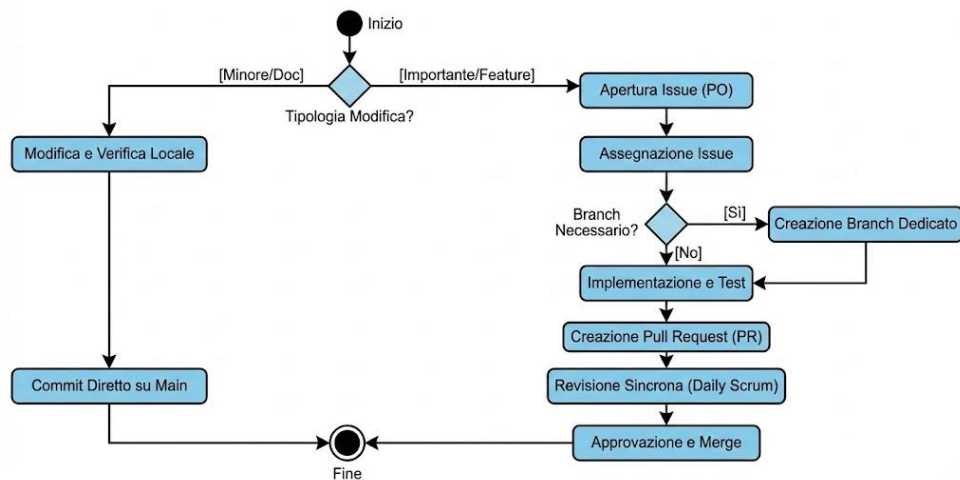
Qui un esempio (in data 03/01/2026):

Todo 5	In Progress 3	Done 24
This item hasn't been started	This is actively being worked on	This has been completed
☒ ProgettoSW25-26 #3 Problema prenotazioni simultanee	☒ ProgettoSW25-26 #5 Implementazione visualizzazione stato carica auto	☒ ProgettoSW25-26 #16 Aggiungere selezione auto in fase di prenotazione
☒ ProgettoSW25-26 #8 Separazione della generazione della mappa rispetto alla lettura delle colonnine	☒ ProgettoSW25-26 #9 Cancellazione Prenotazioni dopo Utenti e/o Colonnine	☒ ProgettoSW25-26 #17 Aggiungere stato nella prenotazione non attiva/attiva/in carica + aggiungere tasto
☒ ProgettoSW25-26 #22 Aggiungere ricerca luoghi sopra la mappa	☒ ProgettoSW25-26 #31 Divisione CSS da Java	☒ ProgettoSW25-26 #11 Implementare sicurezza pagine
☒ ProgettoSW25-26 #24 Sistemare dialog nuova colonnina in AdminColonnineView		☒ ProgettoSW25-26 #12 Controllo prenotazione passata/futura per qrcode
☒ ProgettoSW25-26 #27 Dividere RegisterView in metodi		☒ ProgettoSW25-26 #1 Errore pulsante registrazione
		☒ ProgettoSW25-26 #2 Aggiunta classe AppShell
+ Add item	+ Add item	+ Add item

Gestione delle change request

1. Modifiche minori: In caso di modifiche minori, refactoring o aggiornamenti a diagrammi o documentazione ciascun membro del gruppo di occupava di modificare il codice e verificare il suo corretto funzionamento. Il commit veniva poi effettuato direttamente sul ramo main.
2. In caso di modifiche più importanti o nuove funzionalità tracciate attraverso issue il flusso seguito era il seguente:
 - Apertura issue da parte del membro del gruppo (in particolare dal product owner)
 - Assegnazione issue o presa in carica da parte del membro del team
 - Apertura branch dedicato (non sempre è stato necessario)
 - Implementazione e test
 - Pull request (con eventuale descrizione)
 - Revisione della richiesta da parte di un altro membro e approvazione

Nota: questa metodologia è stata utilizzata un numero limitato di volte, data la dimensione ridotta del team e dati i frequenti meeting in cui il team verificava direttamente le modifiche esposte dal membro che aveva preso in carico la issue o il problema da risolvere. Gran parte delle revisioni sono dunque avvenute tramite revisione sincrona durante i Daily Scrum.



Statistiche della repo (al 29/01/2026)

- 1- Numero totale di commit: 250
- 2- Numero di issue gestite: 35
- 3- Branch utilizzati: 9
- 4- Numero di pull request: 7
- 5- Contributors: 4 (ciascuno con il proprio ruolo)

People Management and Team Organization

Ruoli

Membro del Team	Ruolo Scrum	Responsabilità e Competenze Specifiche
Davide Mascheretti	Product Owner	Backend Developer
Tommaso Maistrello	Scrum Master	Frontend Developer
Francesco Valenari	Development Team	Frontend
Claudio Morgera	Development Team	Backend

Comunicazione

La comunicazione nel team è avvenuta attraverso differenti metodi

- 1- In presenza, per planning e review
- 2- In presenza/tramite messaggistica: Daily Scrum
- 3- Whatsapp: comunicazione giornaliera rapida
- 4- Github: documentazione, issues e commenti